

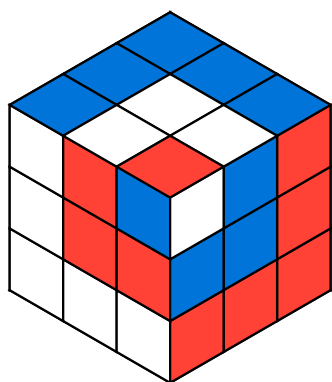
# magic-cubes

v0.0.1      MIT

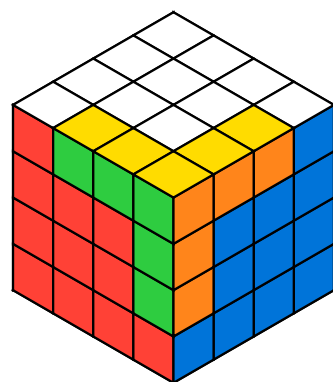
Represent Rubik's cubes of arbitrary size and apply algorithms to them

Rodrigo Alcántara

 @RodAlc24



U' L' U' F' R2 B' R  
F U B2 U B' L U' F  
U R F'



F U2 L F L' B L U  
B' R' L' U R' D' F'  
B R2

## Table of Contents

- About ..... 2
- Guide ..... 3
  - II.1 Creating cubes ..... 3
  - II.2 Applying algorithms ..... 3
    - II.2.1 Moves ..... 3
    - II.2.2 Modifiers ..... 3
  - II.3 Rendering cubes ..... 3
- Full reference ..... 4
  - III.1 Types ..... 4
  - III.2 Functions ..... 4
- Index ..... 8

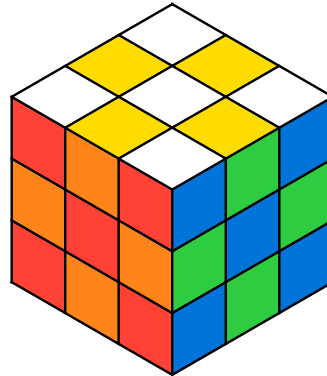
# Part I

## About

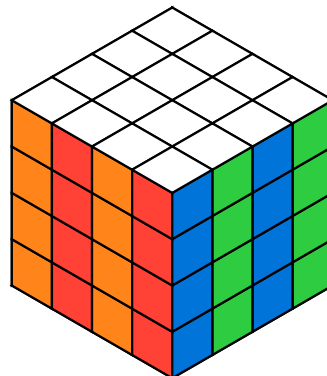
In 1974, Ernő Rubik invented a mechanical puzzle and called it the *Magic Cube*. Years later, in 1980, the puzzle was renamed the *Rubik's Cube*, name by which it is now known. Today, it is considered to be the world's bestselling puzzle game.

[magic-cubes](#) is a package built on top of [CeTZ](#) that allows you to create, manipulate and render Rubik's cubes of any size.

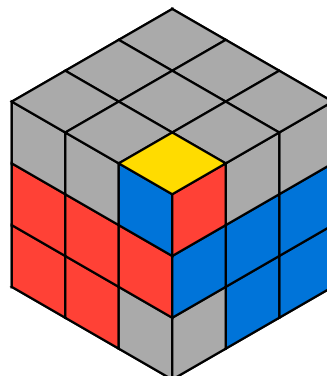
```
#draw_cube(  
  apply(  
    cube(),  
    "F2 B2 R2 L2 U2 D2"  
  )  
)
```



```
#draw_cube(  
  apply(  
    cube(size: 4),  
    "R2 3R2 F2 3F2 R2 3R2"  
  )  
)
```



```
#draw_cube(  
  apply(  
    inverted: true,  
    f2l-cube,  
    "R2 U R2 U R2 U2 R2"  
  )  
)
```



# Part II

## Guide

### II.1 Creating cubes

Cubes can be created using the “cube” function...

### II.2 Applying algorithms

#### II.2.1 Moves

The following moves are supported, more will be added in future updates:

- F: front face
- R: right face
- U: up face
- B: back face
- L: left face
- D: down face

All of them represent a single **clockwise** turn in the respective face.

#### II.2.2 Modifiers

Adding an apostrophe (') or a “2” after the move will cause an anticlockwise turn or a double one (180°) respectively.

### II.3 Rendering cubes

# Part III

## Full reference

### III.1 Types

```
(  
  {f} dictionary  
  {r} dictionary  
)
```

### III.2 Functions

`#apply` `#draw_cube` `#draw_flat`

**#apply**(({cube}), {alg}, {inverted}): false → cube-t

Returns a new cube after applying the algorithm.

Argument —  
{cube} cube-t  
The cube to apply the algorithm.

Argument —  
{alg} str  
The algorithm to apply.

Argument —  
{inverted}: false bool  
Whether it should be applied in inverse order.

**#draw\_cube**(({cube}), {x}: 35.264deg, {y}: 45deg, {z}: 0deg)

Draws a cube

Argument —  
{cube} cube-t  
the cube to draw.

Argument —  
{x}: 35.264deg angle  
the x angle.

Argument —  
{y}: 45deg angle

the y angle.

Argument

`{ z } : 0deg`

angle

the z angle.

**#draw\_flat({cube})**

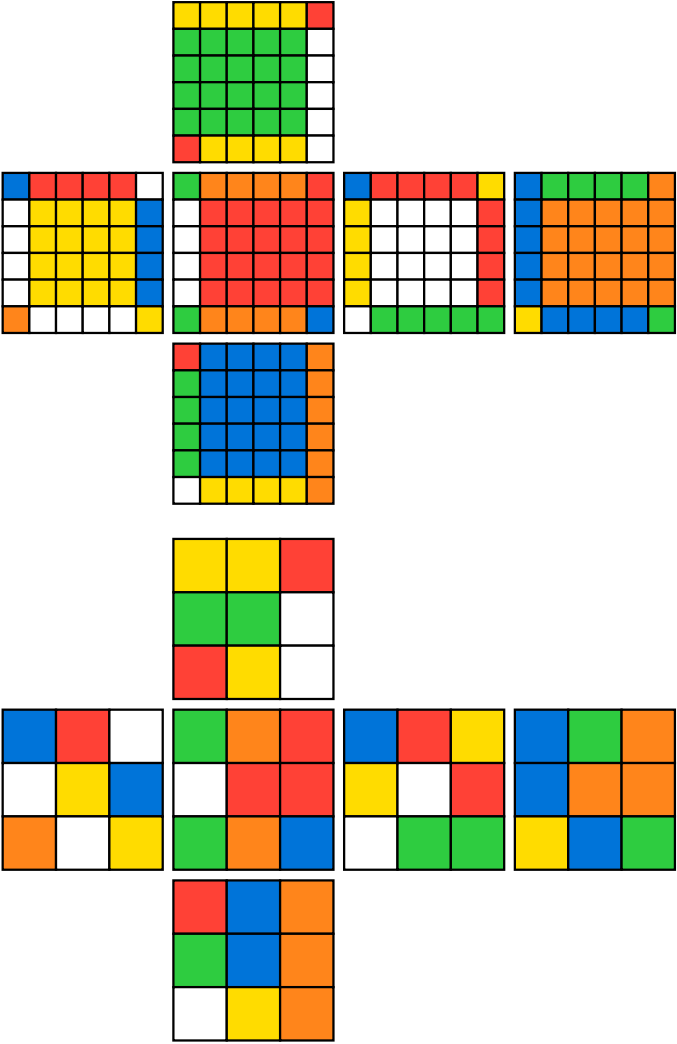
Draws a flat cube.

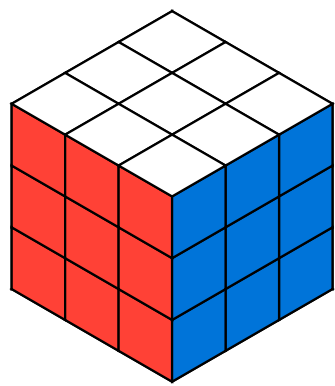
Argument

`{cube}`

cube - t

The cube to draw.





,

# Part IV

## Index

### A

`#apply` ..... 4

### C

`cube` ..... 4

### D

`#draw_cube` ..... 4

`#draw_flat` ..... 4, 5